

CSC4207 Group Project Report: GridScript

Group 1: Nabil Ismail Abdulkadir (UG22CSC1047), Ahmad Auwal Abubakar (UG22CSC1075), Abubakar Muhammad Sulaiman (UG22CSC1046), Rukayya Musbahu Imam (UG22CSC1040), Muhammad Salisu (UG20CSC1005)

What we built

GridScript is a small programming language for scripting a game actor on a 2D grid, and an interpreter for it, written in Python. A GridScript program can set variables, do arithmetic, run if and while blocks, define functions with parameters and return, and call built-in actions like `step_forward()`. We kept the language small on purpose. The brief warned that depth of understanding matters more than feature count, and two weeks is not much time.

The interpreter runs in three stages, and there is no compile step. `gridscript.py` reads a `.gsc` file and evaluates it directly, which is how the intro slides (01) define an interpreter: it executes each instruction in the source.

How the interpreter works

The lexer (`src/lexer.py`) reads the source one character at a time and turns it into tokens. Each token type is a regular expression built from the three operations in the automata slides (02A): concatenation, union, and Kleene closure, with star binding tightest. Numbers are `[0-9]+`, identifiers are `[a-zA-Z_][a-zA-Z0-9_]*`, a union of ranges followed by a closure. Twelve reserved words are checked in a keyword table before a word becomes an identifier. The lexer always takes the longest match, so `set` becomes a keyword and never a variable name. Comments after `//` are stripped here. Errors surface early too: a stray `$` never reaches the parser. Figure 2 shows the token stream for the patrol program (`report/vscode_shots/06_tokens_demo.png`).

The parser (`src/parser.py`) is a recursive descent parser written straight from our BNF grammar, one function per grammar rule. The parsing slides define a derivation as a sequence of rewrites and parsing as discovering that derivation; ours does it top-down. Precedence lives in the rule structure across four levels, from comparison down to factor, so `2 + 3 * 4` parses as `2 + (3 * 4)` without any extra logic. The output is an abstract syntax tree of small dataclasses. Figure 3 shows the tree for a function definition (`report/vscode_shots/07_ast_demo.png`).

The evaluator (`src/interpreter.py`) walks that tree and applies one rule per construct. We wrote these rules out in the design document as inference rules before implementing them, in the same style as the semantics slides (04). An assignment evaluates the expression and binds the name in the current environment. A function call creates a fresh environment, binds each parameter to its argument, and runs the body until a return carries the value back to the caller. Figure 4 shows the patrol program running end to end (`report/vscode_shots/03_patrol_demo.png`).

Scoping and typing choices

We chose static (lexical) scoping with dynamic typing.

Only function calls create a new scope. `set x = 5` always writes to the current environment, while reading a name walks outward through the enclosing environments. This matches the semantics slides, where an environment is a partial function from identifiers to values, undefined for names never bound. The consequence is easy to show: a function that assigns `x` cannot touch the global `x`. Our shadowing test prints 100, then 7, then 100 again (Figure 5, `report/vscode_shots/04_shadowing_demo.png`). We picked static scoping because you can understand a function by reading its text alone. With dynamic scoping its behaviour would depend on who called it.

Each function also stores the environment where it was defined, which is exactly the closure definition from the functional programming slides (05): a closure is a function plus its environment.

Values carry their type at runtime: Number, String, Boolean. The language has no type annotations, so checks happen when operations run. This is the dynamic typing definition from the types slides (06): types decided while the program runs, values carrying tags, disallowed operations raising runtime exceptions. + accepts two numbers or two strings, and mixing them raises a type error. Conditions must be Booleans, so if 5 then ... is rejected at runtime. We considered accepting any value as a condition but decided strictness was easier to explain and easier to test. Figure 6 shows an invalid program being rejected with its exact message (report/vscode_shots/10_runtime_error.png).

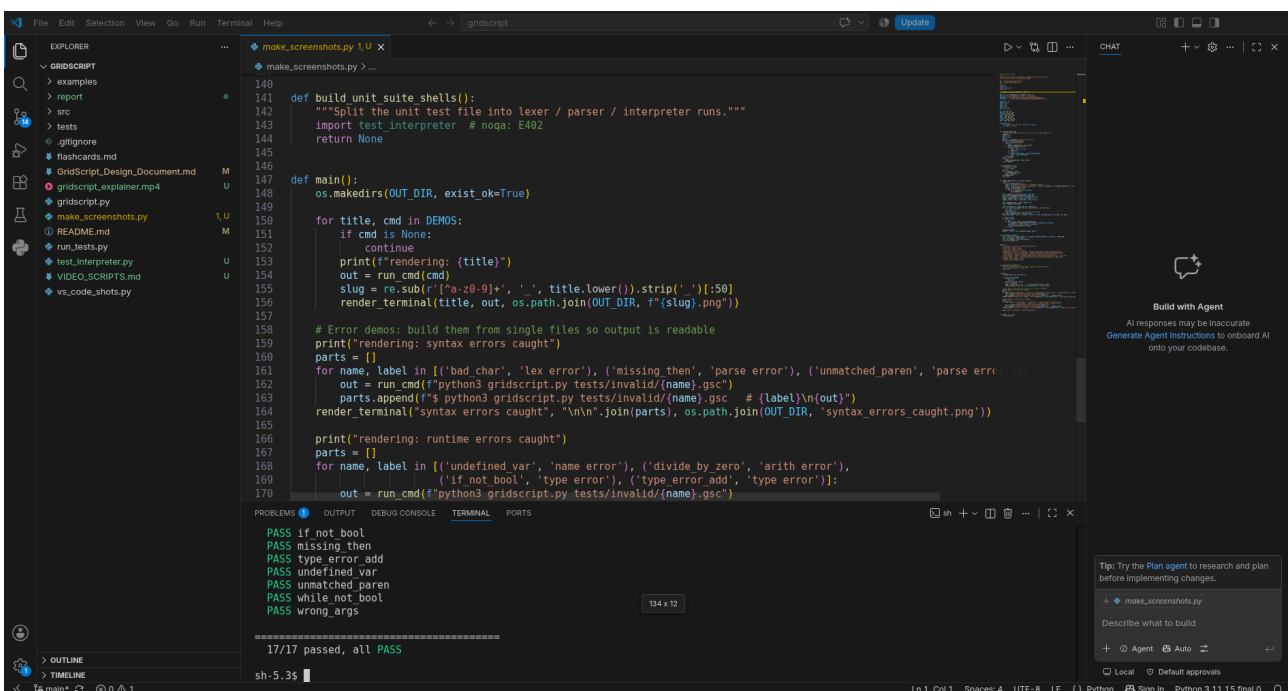
Testing

Salisu built two test layers, and both are green. run_tests.py runs whole programs: valid ones are compared against expected output, and nine invalid programs must each fail with a specific message, whether a lexer error, a parser error, or a runtime type error. Figure 1 shows the full integration suite passing, 17 of 17 (report/vscode_shots/01_full_test_suite.png). test_interpreter.py holds 61 unit tests covering the lexer, parser, evaluator, scoping and typing separately. We also checked that the suite catches regressions: changing one expected output file produces a red FAIL with a diff (report/vscode_shots/11_failed_test_caught.png), which we then reverted.

What we used AI for

The brief allows AI for minor tasks. We used NotebookLM to brainstorm language themes and to produce explainer slides and a video for our first group meeting. AI also helped debug one Python error message during development. The grammar, the scoping and typing rules, and all core code were written and reviewed by the group, and every member explains their own part in the walkthrough videos.

Appendix A: test evidence



```
def build_unit_suite shells():
    """Split the unit test file into lexer / parser / interpreter runs."""
    import test_interpreter # noqa: E402
    return None

def main():
    os.makedirs(OUT_DIR, exist_ok=True)

    for title, cmd in DEMOS:
        if cmd is None:
            print(f"rendering: {title}")
            out = run_cmd(cmd)
            slug = re.sub(r'[^a-z0-9]+', '-', title.lower()).strip('.')[:50]
            render_terminal(title, out, os.path.join(OUT_DIR, f"{slug}.png"))

    # Error demos: build them from single files so output is readable
    print("rendering: syntax errors caught")
    parts = []
    for name, label in [
        ('bad char', 'lex error'), ('missing then', 'parse error'), ('unmatched_paren', 'parse error')
    ]:
        out = run_cmd(f"python3 gridscript.py tests/invalid/{name}.gsc")
        parts.append(f"$ python3 gridscript.py tests/invalid/{name}.gsc # {label}\n{out}")
    render_terminal("syntax errors caught", "\n\n".join(parts), os.path.join(OUT_DIR, 'syntax_errors_caught.png'))

    print("rendering: runtime errors caught")
    parts = []
    for name, label in [
        ('undefined var', 'name error'), ('divide by zero', 'arith error'),
        ('if not bool', 'type error'), ('type error add', 'type error')
    ]:
        out = run_cmd(f"python3 gridscript.py tests/invalid/{name}.gsc")
        parts.append(f"$ python3 gridscript.py tests/invalid/{name}.gsc # {label}\n{out}")
    render_terminal("runtime errors caught", "\n\n".join(parts), os.path.join(OUT_DIR, 'runtime_errors_caught.png'))

if __name__ == '__main__':
    main()
```

```
PASS if not bool
PASS missing then
PASS type error add
PASS undefined var
PASS unmatched_paren
PASS while not bool
PASS wrong_args

=====
17/17 passed, all PASS
sh-5.3$
```

Figure 1: the full integration suite passing, 17 of 17.

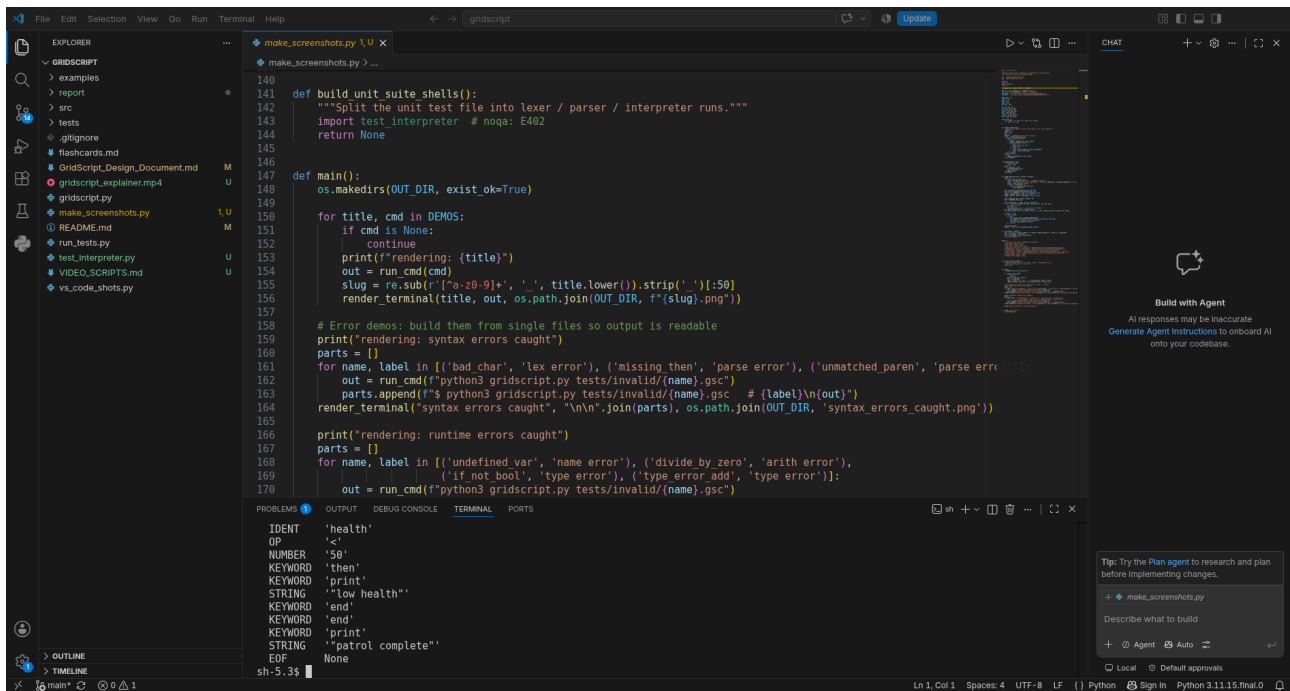


Figure 2: lexer token output for the patrol program.

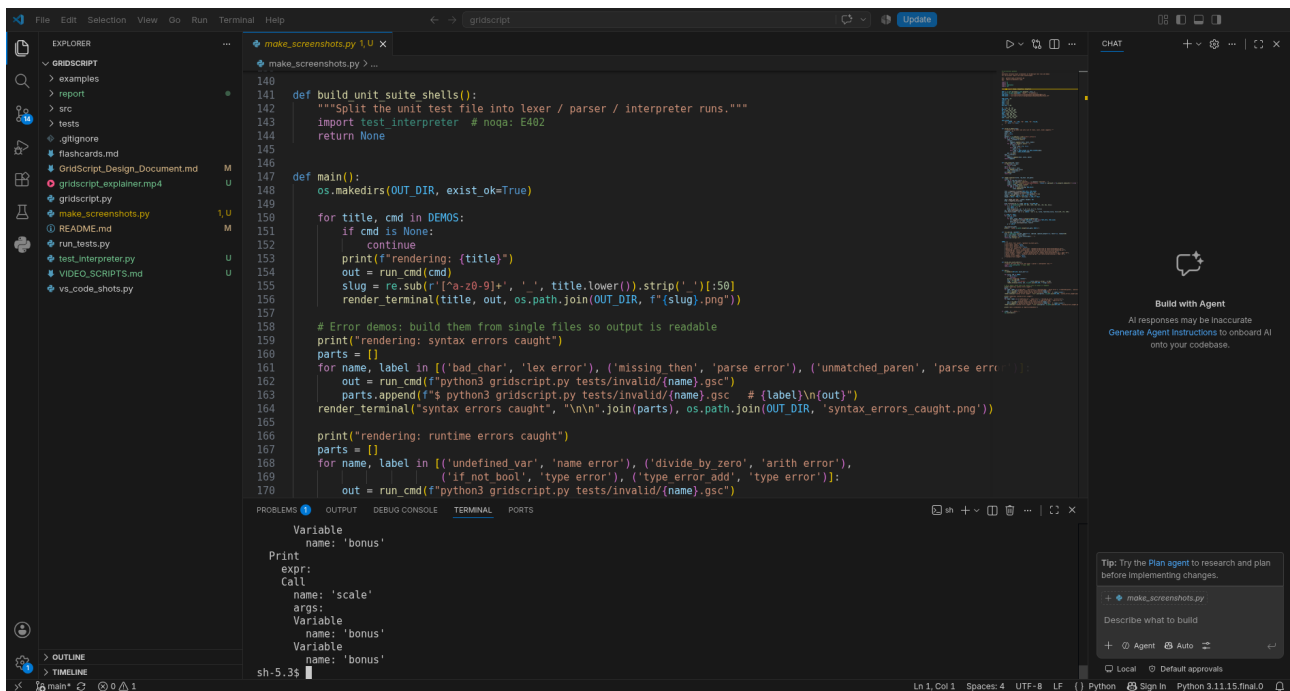


Figure 3: parser AST output for a function definition.

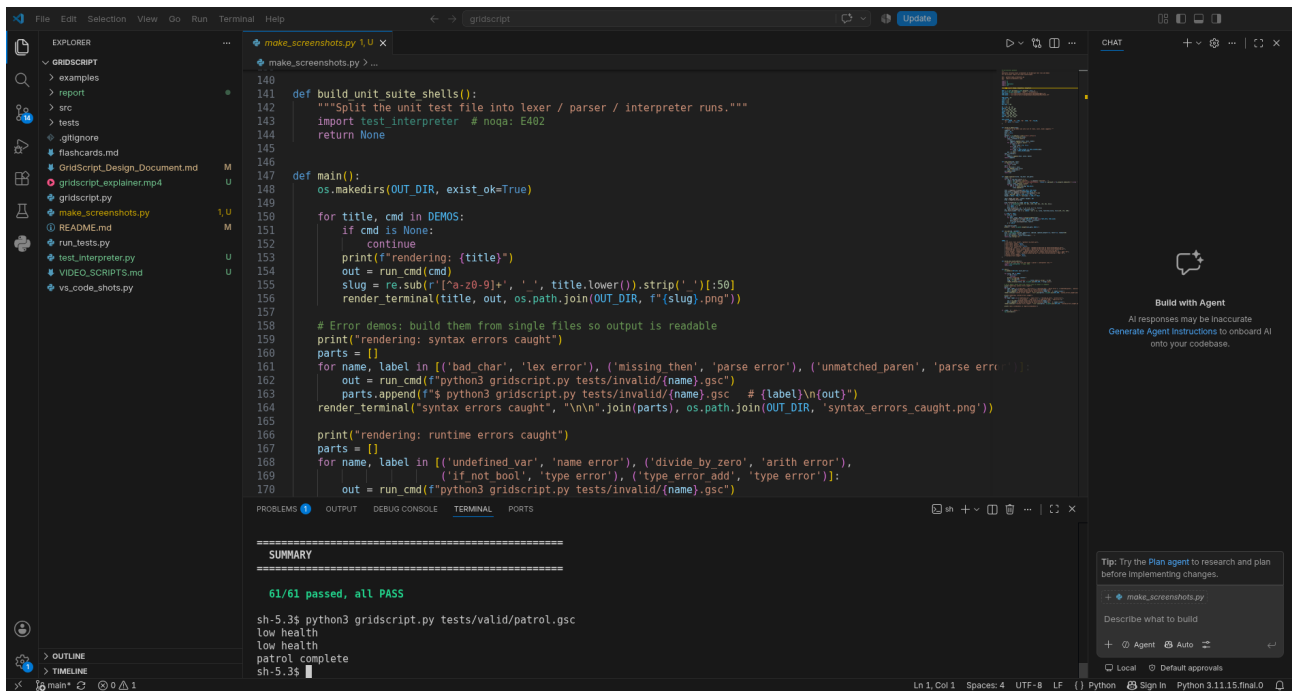


Figure 4: the patrol program running end to end.

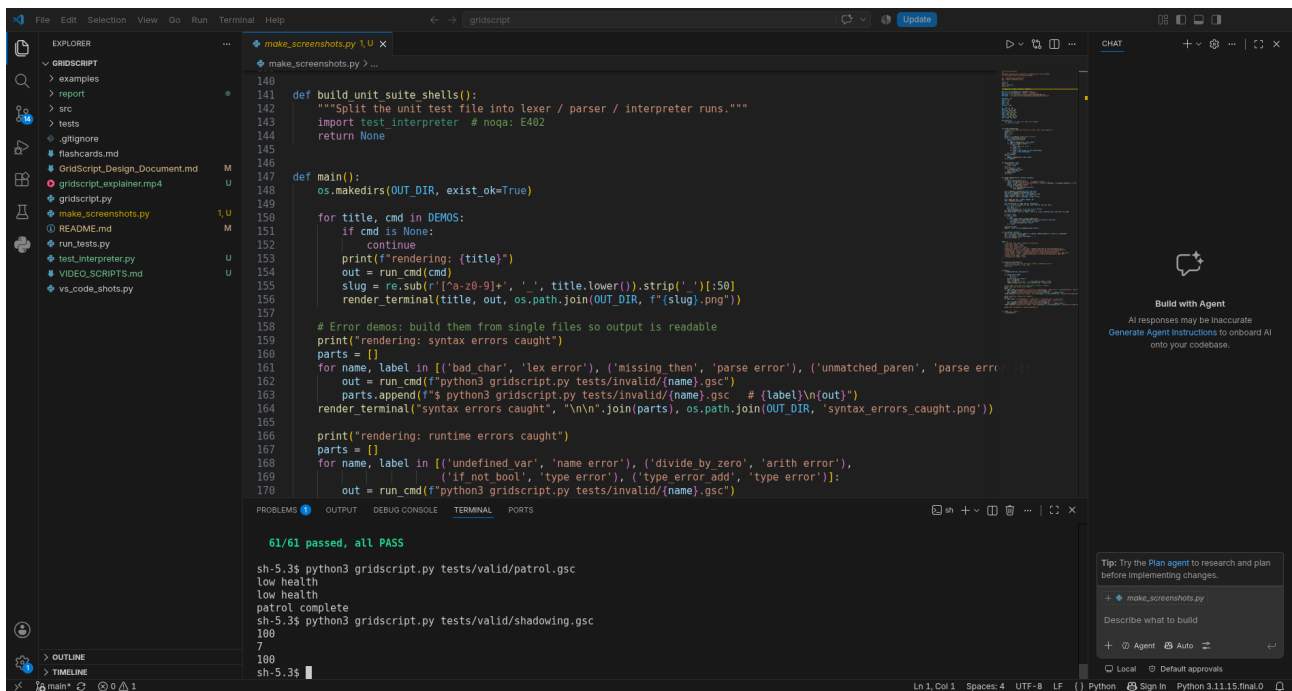


Figure 5: static scoping proof, 100, 7, 100.

